

Laborator 12

Maparea fișierelor în memorie și memoria partajată

12.1 Scopul lucrării

Laboratorul curent va prezenta principalele concepte legate de maparea fișierelor în memorie și memoria partajată, precum și apelurile de sistem Linux, în varianta POSIX pentru lucrul cu aceste concepte.

12.2 Fundamente teoretice

12.2.1 Maparea fișierelor în memorie

Operațiile comune pe fișiere (**read** și **write**) ajută la accesarea secvențială a unui fișier (citirea sau scrierea datelor “în ordine”). Atunci când dorim acces aleator la un fișier (vrem să accesăm date de la adrese arbitrare) putem folosi apelul de sistem **lseek**, dar apelul frecvent al acestuia șiținerea evidenței poziției cursorului pot complica un program.

Maparea unui fișier în memorie simplifică operațiile asupra acestuia, oferind utilizatorului acces aleator la nivel de octet, ca și cum întreg fișierul ar fi fost citit în memorie. În realitate, spațiul este doar rezervat în memoria virtuală, urmând să se facă citiri doar atunci când datele sunt accesate.

12.2.2 Memoria partajată

Memoria partajată este un mecanism de comunicare între procese, care permite existența unei zone de memorie comună ambelor procese.

În condiții normale, thread-urile împart spațiul de adrese din procesul în care rulează, în timp ce procesele au spații de adrese diferite, pentru a fi protejate unele de altele. Totuși, există situații când dorim ca două procese să comunice împărțind o zonă de memorie comună.

Figura 12.1 ilustrează modul în care sistemul de operare implementează conceptul de memorie partajată. Fiecare proces are la dispoziție un spațiu de adrese, din care anumite pagini sunt utilizate, având un corespondent în memoria fizică. Legătura dintre memoria virtuală și memoria fizică se face transparent, un proces neputând adresa memorie din afara spațiului său de adrese. Chiar dacă două procese P_1 și P_2 folosesc aceleași adrese de memorie virtuală, sistemul de operare le va asocia în mod normal cu adrese diferite din memoria fizică. Doar dacă procesele cer explicit memorie partajată, paginile acestora vor fi asociate cu aceeași zonă de memorie fizică. În acest caz, o scriere în memorie partajată efectuată de P_1 va fi văzută automat de P_2 .

12.2.3 Lucrul cu fișiere mapate în memorie

```
#include <sys/mman.h>
```

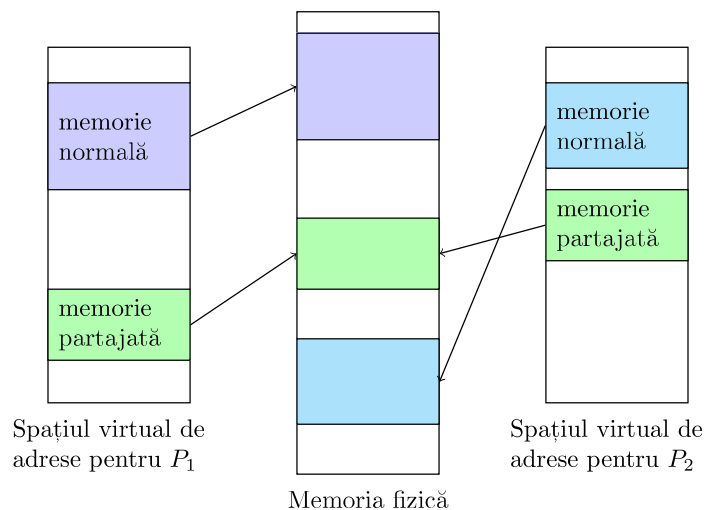


Figura 12.1: Modul în care sistemul de operare implementează conceptul de memorie partajată

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);
```

12.2.3.1 mmap

Apelul de sistem `mmap` ne permite maparea în memorie a unui fișier sau chiar obținerea unei zone de memorie partajată cu alt proces.

Pentru a mapa un fișier în memorie, trebuie să îl deschidem întâi folosind apelul de sistem `open`, descriptorul de fișier obținut fiind transmis ca al 5-lea argument funcției `mmap`. Primul argument al funcției, `addr` poate fi `NULL`, caz în care sistemul de operare alege adresa unde se va realiza maparea sau se poate forța o adresă specificată.

`mmap` ne permite să nu mapăm neapărat tot fișierul, putând să specificăm un offset, ca ultim argument, respectiv un număr de octeți care trebuie mapați, ca argumentul 2.

Argumentul al treilea, `prot` specifică protecția paginilor mapate (adică modul în care memorie poate fi accesată), care poate fi `PROT_NONE` sau o combinație dintre flag-urile `PROT_READ`, `PROT_WRITE` și `PROT_EXECUTE`. Protecția memoriei mapate trebuie să fie în acord cu modul în care fișierul a fost deschis, adică, de exemplu, dacă protecția este `PROT_READ`, modul de deschidere a fișierului trebuie să fie `O_RDONLY` sau `O_RDWR`.

Al 4-lea argument, `flags`, trebuie să fie una din valorile `MAP_SHARED` și `MAP_PRIVATE` în combinație cu anumite flag-uri specifice operației de mapare. `MAP_SHARED` indică faptul că maparea poate fi partajată cu alte procese, iar în cazul în care un fișier este mapat pentru scriere, modificările vor ajunge pe disc. `MAP_PRIVATE` nu permite partajarea mapării cu alte procese și nu propagă modificările la fișier.

Funcția `mmap` returnează un pointer la adresa unde s-a făcut maparea, în caz de succes, sau `(void*)-1`, în caz de eșec.

12.2.3.2 munmap

Acest apel de sistem închide o mapare obținută cu `mmap`. Primul argument reprezintă adresa în memorie unde s-a realizat maparea, în timp ce al doilea conține dimensiunea mapării.

12.2.3.3 Exemplu

Exemplul de mai jos va afișa conținutul unui fișier invers, folosind tehnica mapării în memorie. Fără această tehnică, soluția ar fi să facem operații de `lseek` și `read` pentru fiecare caracter. O altă soluție ar fi fost să citim tot fișierul într-un vector în memorie, apoi să parcurgem invers vectorul

citit. A doua soluție ar fi mai ușor de scris, dar pentru fișiere mari am avea un consum mare de memorie.

Tehnica mapării în memorie ne oferă acces la fișier ca și cum acesta ar fi fost citit în întregime, dar realizând citirea în mod transparent, doar atunci când datele sunt necesare.

```

1  int main(int argc, char **argv)
2  {
3      int fd;
4      off_t size, i;
5      char *data = NULL;
6
7      if(argc != 2) {
8          fprintf(stderr, "Usage: %s <file_name>\n", argv[0]);
9          return 1;
10     }
11
12     fd = open(argv[1], O_RDONLY);
13     if(fd == -1) {
14         perror("Could not open input file");
15         return 1;
16     }
17     size = lseek(fd, 0, SEEK_END);
18     lseek(fd, 0, SEEK_SET);
19
20     data = (char*)mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
21     if(data == (void*)-1) {
22         perror("Could not map file");
23         close(fd);
24         return 1;
25     }
26
27     for(i=size-1; i>=0; i--) {
28         printf("%c", data[i]);
29     }
30     printf("\n");
31
32     munmap(data, size);
33     close(fd);
34
35     return 0;
36 }
```

Programul începe prin verificarea numărului de argumente primite în linia de comandă (linia 7), apoi se realizează deschiderea fișierului în citire (linia 12). Modul de deschidere a fișierului trebuie să corespundă modului în care vom accesa pe viitor memoria partajată. În cazul acesta am deschis în citire, deoarece vrem doar să citim datele, fără să facem modificări.

Următorul pas este aflarea dimensiunii fișierului apelând `lseek` pentru a ajunge la finalul acestuia (linia 17). Dimensiunea fișierului se poate afla și folosind apelul de sistem `fstat`.

Urmează maparea fișierului în memorie la linia 20, unde folosim descriptorul de fișier `fd` și dimensiunea obținută anterior `size`. Lăsăm sistemul de operare să decidă adresa mapării, transmitând `NULL` ca prim argument și cerem ca paginile mapate să fie accesibile doar în citire (`PROT_READ`). Nu dorim să partajăm maparea, nici să salvăm modificări asupra fișierului, deci alegem modul `MAP_PRIVATE`. Funcția `mmap` returnează un pointer la începutul memoriei mapate, pe care îl salvăm în variabila `data`.

Având la dispoziție memoria mapată, parcurgea în ordine inversă a fișierului se rezumă la o simplă buclă `for` descrescătoare (liniile 27–29).

În final, trebuie să închidem maparea (linia 32) și fișierul (linia 33). Închiderea fișierului poate

fi făcută și imediat după mapare, deoarece funcția `mmap` creează în mod intern un duplicat al descriptorului de fișier.

12.2.4 Lucrul cu memoria partajată

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>

int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);

int ftruncate(int fd, off_t length);
```

Atunci când utilizăm memoria partajată, trebuie să cerem compilatorului (componentei *linker*) să adauge biblioteca *rt*, adăugând în linia de comandă opțiunea `-lrt`.

Compilarea programelor care folosesc memoria partajată <code>gcc -Wall program.c -o program -lrt</code>
--

12.2.4.1 shm_open

Apelul de sistem `shm_open` creează sau deschide o zonă de memorie partajată. Funcționează în mod similar cu apelul de sistem `open` pentru fișiere, returnând un descriptor de fișier, care apoi se poate mapa în memorie cu `mmap`.

Primul parametru, `name`, este numele memoriei partajate. Dacă două procese apelează `shm_open` cu același nume, vor primi acces la aceeași zonă de memorie. Standardul POSIX recomandă ca numele unei memorii partajate să înceapă cu caracterul `'/'`, să nu mai conțină alte caractere `'/'` în afară de primul și să aibă maxim 255 de caractere.

Parametrul `oflag` poate fi `O_RDONLY` (citire) sau `O_RDWR` (citire și scriere), combinat prin SAU logic cu alte flag-uri, precum `O_CREAT`, care indică crearea memoriei partajate, în caz că nu există.

Parametrul `mode` conține permisiunile memoriei partajate (similare cu permisiunile unui fișier) și este luat în considerare doar când memoria se creează.

În caz de eșec se returnează o valoare negativă.

12.2.4.2 shm_unlink

Acest apel de sistem realizează ștergerea unei zone de memorie partajată, primind ca parametru numele acesteia (același nume care a fost folosit și la `shm_open`).

După ce se apelează `shm_unlink` memoria partajată nu se mai poate deschide (decât dacă se creează din nou), dar procesele care o mapează o pot accesa în continuare, ștergerea efectivă efectuându-se doar după ce ultimul proces a închis maparea.

12.2.4.3 ftruncate

Acest apel de sistem funcționează atât pentru fișiere obișnuite cât și pentru zone de memorie partajată și are rolul de a redimensiona fișierul cu descriptorul `fd`, la noua dimensiune `length`.

În cazul unei zone de memorie partajată, e important să apelăm `ftruncate` după ce aceasta a fost creată, pentru a defini dimensiunea acesteia.

12.2.4.4 Exemplu

Vom prezenta un exemplu simplu, în care două procese partajează o zonă de memorie de un singur octet. Primul proces creează zona de memorie partajată și scrie în ea un anumit caracter, apoi așteaptă ca un alt proces să îl modifice.

Atunci când declarăm în C un pointer către o zonă de memorie partajată (linia 3) se recomandă utilizarea cuvântului-cheie `volatile`, care indică compilatorului faptul că acea variabilă se poate modifica în moduri neprevăzute (de exemplu de către un alt proces cu care se partajează memoria). Operațiile pe variabila respectivă nu vor fi optimizate și vor fi efectuate precum în codul sursă.

p1.c:

```

1 | int main(void) {
2 |     int shmFd;
3 |     volatile char *sharedChar = NULL;
4 |
5 |     shmFd = shm_open("/l12_myshm", O_CREAT | O_RDWR, 0600);
6 |     if(shmFd < 0) {
7 |         perror("Could not acquire shm");
8 |         return 1;
9 |     }
10 |    ftruncate(shmFd, sizeof(char));
11 |    sharedChar = (volatile char*)mmap(0, sizeof(char),
12 |        PROT_READ | PROT_WRITE, MAP_SHARED, shmFd, 0);
13 |    if(sharedChar == (void*)-1){
14 |        perror("Could not map the shared memory");
15 |        return 1;
16 |    }
17 |
18 |    *sharedChar = 'A';
19 |    while(*sharedChar == 'A') {
20 |        printf("sharedChar: %c\n", *sharedChar);
21 |        sleep(1);
22 |    }
23 |    printf("sharedChar new value: %c\n", *sharedChar);
24 |
25 |    munmap((void*)sharedChar, sizeof(char));
26 |    sharedChar = NULL;
27 |    close(shmFd);
28 |    shm_unlink("/l12_myshm");
29 |
30 |    return 0;
31 | }
```

Primul program, *p1*, are rolul de crea zona de memorie partajată, apelând `shm_open` la linia 5. Numele zonei de memorie `"/l12_myshm"` este folosit și de *p2*, pentru a lucra cu aceeași zonă de memorie. După ce obținem descriptorul de fișier `shmFd`, trebuie să apelăm `ftruncate` pentru a stabili dimensiunea zonei de memorie (linia 10), în acest caz 1 byte (`sizeof(char)`).

În continuare, zona de memorie partajată va fi mapată în spațiul curent de adrese, folosind `mmap` (liniile 11–12). Spre deosebire de exemplul din secțiunea 12.2.3.3, memoria mapată trebuie să fie partajată (folosim `MAP_SHARED`), iar protecția paginilor să fie atât `PROT_READ` cât și `PROT_WRITE`. Rezultatul operației este un pointer pe care îl vom stoca în variabila `sharedChar`.

La linia 18 se setează conținutul memoriei partajate la litera 'A', apoi urmează o buclă în care se verifică dacă acest conținut s-a modificat (liniile 19–22). După modificare, se afișează noua valoare (linia 23), apoi se încheie maparea (linia 25), descriptorul `shmFd` (linia 27) și se șterge zona de memorie partajată folosind apelul de sistem `shm_unlink` (linia 28).

După închiderea mapării, pointer-ul `sharedChar` încă pointează la vechea adresă, care acum este invalidă. Este o practică bună în C ca pointerii rămași invalizi să fie setați la `NULL`, astfel încât la reutilizarea acestora necorespunzătoare să nu corupem date.

p2.c:

```

1 | int main(void) {
2 |     int shmFd;
```

```

3     volatile char *sharedChar = NULL;
4
5     shmFd = shm_open("/l12_myshm", O_RDWR, 0);
6     if(shmFd < 0) {
7         perror("Could not acquire shm");
8         return 1;
9     }
10    sharedChar = (volatile char*)mmap(0, sizeof(char),
11        PROT_READ | PROT_WRITE, MAP_SHARED, shmFd, 0);
12    if(sharedChar == (void*)-1){
13        perror("Could not map the shared memory");
14        return 1;
15    }
16
17    printf("found sharedChar: %c\n", *sharedChar);
18    *sharedChar = 'X';
19
20    munmap((void*)sharedChar, sizeof(char));
21    sharedChar = NULL;
22    close(shmFd);
23
24    return 0;
25 }

```

Programul *p2* accesează memoria partajată creată de *p1*, folosind aceeași valoare pentru parametrul *name* în apelul funcției `shm_open` (linia 5). Modul de deschidere va fi `O_RDWR`, iar permisiunile au valoarea 0, deoarece apelul curent nu creează nimic, doar obține descriptorul zonei de memorie ce a fost creată în prealabil de *p1*. Deasemenea, nu mai e nevoie să apelăm `ftruncate`, deoarece dimensiunea memorie partajate a fost deja stabilită.

În mod similar cu *p1*, mapăm zona de memorie (liniile 10–11), obținând pointer-ul `sharedChar`, a cărui valoare o afișăm (linia 17) apoi o setăm la caracterul ‘X’ (linia 18). Programul se termină prin închiderea mapării (linia 20), setarea pointer-ului la `NULL` (linia 21) și închiderea descriptorului de fișier (linia 22).

Vom rula întâi programul *p1*, care creează zona de memorie, apoi așteaptă în buclă modificarea caracterului din aceasta. Putem observa că o dată pe secundă, valoarea este afișată. Într-un alt terminal, rulăm *p2* și observăm că acesta găsește caracterul ‘A’ în memoria partajată, apoi îl modifică. Între timp, dacă revenim la terminalul în care am rulat *p1*, observăm că după rularea lui *p2* valoarea caracterului s-a modificat, iar programul a ieșit din buclă și s-a încheiat.

Observație: Cele două programe vor rula ca mai sus doar dacă al doilea program este pornit după primul. Altfel, cel de-al doilea program nu va găsi memoria partajată și își va încheia execuția afișând un mesaj de eroare la linia 7. O implementare robustă ar putea folosi semafoare cu nume pentru a sincroniza cele două programe, inclusiv accesul la memoria partajată.

12.3 Mersul lucrării

12.3.1 Întrebări recapitulative

1. Ce returnează funcția `mmap`?
2. Care este avantajul de a mapa un fișier în memorie în loc să îl citim folosind `read`?
3. Cu ce apel de sistem deschidem o zonă de memorie partajată?
4. Ce face apelul de sistem `ftruncate` și la ce e util în lucrul cu memoria partajată?
5. Dați un exemplu de situație când se preferă comunicarea a două procese prin memorie partajată în loc de pipe-uri.

6. Ce beneficii aduce comunicarea prin pipe-uri în loc de memoria partajată?
7. Cum putem sincroniza două procese diferite utilizând un lacăt (mutex)?

12.3.2 Probleme rezolvate

1. (*l12p1_invert.c*) Să se scrie un program care afișează conținutul unui fișier pe ecran, inversat, folosind tehnica mapării în memorie. Programul este descris în secțiunea 12.2.3.3.
2. (*l12p2_shm1.c* și *l12p2_shm2.c*) Să se scrie două programe care comunică prin memorie partajată. Primul program va scrie un caracter în zona de memorie partajată, apoi va aștepta ca al doilea program să îl modifice. Programele sunt descrise în secțiunea 12.2.4.4.

12.3.3 Probleme propuse

1. (25p) Modificați exemplul din secțiunea 12.2.3.3 astfel încât conținutul fișierului să fie inversat pe disc, în loc să se afișeze pe ecran.
2. (25p) Implementați două programe (sau un program care pornește și un proces copil) care incrementează în mod alternativ o variabilă întreagă aflată într-o zonă de memorie partajată.
3. (25p) Utilizând tehnica mapării în memorie, scrieți un program care șterge toate vocalele dintr-un fișier, umplând spațiul liber rămas la finalul fișierului cu caractere ' ' (spații).
4. (35p) Implementați un program care caută numere prime într-un interval primit ca argumente în linia de comandă și le adaugă la finalul unui vector aflat într-o zonă de memorie partajată. Vectorul va avea pe prima poziție numărul de elemente scrise până în prezent, urmate de elementele propriu-zise. Scrierea în memoria partajată trebuie sincronizată cu un semafor. Implementați de asemenea un program *manager* care inițializează memoria partajată și vectorul din aceasta, apoi împarte un interval primit la N procese copii, care vor căuta numerele și le vor scrie în memorie. La final, *manager*-ul va afișa toate numerele din vector.

